

How the FPL System Actually Works

A plain-language technical walkthrough

Written for a reader with a business/finance background, no programming assumed.

1 The bird's-eye view

Fantasy Premier League is, structurally, a portfolio management problem. Every week you hold a portfolio of 15 assets (players), each with a price, and each paying an uncertain weekly dividend (points). You have a fixed budget, position limits, diversification rules (max 3 players per real club), and transaction costs (transfers beyond the first cost you 4 points). Your job is to maximize total dividends over the season.

Any investment process like this splits into two distinct jobs, and so does this system:

1. **The forecaster** — the “research department.” For every player, every week, it estimates how many points he will score in the next few gameweeks. This is a statistical/machine-learning problem.
2. **The optimizer** — the “portfolio construction desk.” Given those forecasts, it works out the best legal squad, which transfers to make, whom to captain, and when to play chips. This is a mathematical optimization problem with a known, exact answer once the forecasts are fixed.

The separation matters because the two jobs fail differently. The forecaster’s world is noisy and probabilistic — it can only be “less wrong.” The optimizer’s world is deterministic — given inputs, it finds the provably best squad, so if the final squads are bad, the fault is almost always the forecasts, not the optimization. Keeping the two apart lets us test and improve each independently.

The whole pipeline, end to end:

raw match data (6 seasons) → cleaned dataset → ~115 predictive features per player-week → per-position forecasting models → predicted points per player per week → MILP optimizer → squad, transfers, captain, chips

Everything below unpacks that chain one link at a time.

2 The data

The raw material is one row per **player per gameweek**: minutes played, goals, assists, bonus points, expected goals (xG — a bookmaker-style estimate of how many goals a player “should” have scored given the quality of his shots), the opponent, home/away, price, ownership, and so on. Six seasons (2020-21 through 2025-26) give roughly **163,000 rows**.

Two sourcing details worth knowing:

- Historical data comes from a well-maintained public archive of FPL statistics (a GitHub repository the community keeps updated). Live data — fixtures for next week, your current squad — comes from FPL’s official API, since the archive obviously has no rows for matches not yet played.

- Gameweeks are numbered on a single continuous counter across all seasons (GW 1-228 so far) rather than resetting to 1 each August. This makes “train on everything before week t ” trivial to express, which, as you’ll see, is the single most important discipline in the whole system.

3 Feature engineering: turning history into predictors

A model cannot eat a player’s biography; it eats numbers. A **feature** is one number per player-week that summarizes something predictive. This system builds ~115 of them per row. The families:

- **Form** — rolling averages of points, minutes, xG, etc. over the last 3 and 5 games, plus two longer horizons: a season-to-date average (resets every August) and a career average. Think of these as the 1-month, 3-month, YTD and since-inception returns on a fund factsheet — same quantity, different memory lengths, each informative about a different thing.
- **Exponentially-weighted form** — like a rolling average, but recent games count more (a half-life of 3 gameweeks: last week’s match counts double a match three weeks ago). The finance analogue is exponentially-weighted volatility (as in RiskMetrics) versus a simple moving window.
- **Efficiency rates** — goals and xG per 90 minutes rather than per game, so a striker who scored in a 20-minute cameo isn’t confused with one who needed the full match.
- **“Nailedness”** — the share of the last 5 games the player started, and the share where he played 60+ minutes. A brilliant player who doesn’t get on the pitch pays no dividend, so predicted minutes is arguably the single most valuable input in fantasy modeling.
- **Opponent strength** — the upcoming opponent’s rolling attacking and defensive form, computed at team level. This is what makes “Haaland at home to the bottom club” and “Haaland away at Arsenal” different forecasts.
- **Fixture difficulty** — FPL’s own published 1-5 difficulty rating for the coming fixture and the average over the next three.
- **FPL’s own forecast (xP)** — FPL publishes its own expected-points number before each gameweek; we feed in the recent history of it, effectively using their model as one input to ours (like including consensus analyst estimates as a feature in your own equity model).

3.1 The one rule that everything depends on: no peeking

Every feature derived from a player’s own performance is **shifted back one gameweek**: the row for gameweek t only ever contains information from gameweeks $t - 1$ and earlier. The forecast for next Saturday cannot contain anything from next Saturday.

This sounds obvious, but it is the modeling equivalent of insider trading, and it creeps in through subtle cracks rather than the front door. In finance backtests this is **look-ahead bias**: a strategy that “knew” tomorrow’s close looks brilliant in-sample and dies in production. Two real examples from this project’s history (both found in an internal review and fixed):

- The live weekly forecast was accidentally reusing each player’s **last played** match row, so its “fixture difficulty” described last week’s opponent, not next week’s.
- The ensemble’s blending weights (explained below) were estimated on the very window the backtest was then scored on — a subtle self-grading loop that inflated the headline backtest by roughly 100 points. The published numbers were re-baselined downward after the fix.

One more data-honesty rule: statistics that simply didn't exist in early seasons (xG wasn't collected before 2022-23) are stored as **missing**, not as zero. "We didn't measure it" and "he generated exactly none" are different facts, and the models treat them differently.

4 The forecasting models

4.1 Why four models, not one

The system trains a separate model per position (GK, DEF, MID, FWD). What predicts a goalkeeper's points (saves, clean sheets) has almost nothing in common with what predicts a forward's (xG, penalty duty). Pooling them into one model forces a single set of rules to serve four different games at once.

4.2 The registry: twelve candidate algorithms and an index to beat

Rather than betting on one algorithm, the project maintains a **registry** of twelve — from plain linear regression up through modern gradient-boosted trees (LightGBM, XGBoost, CatBoost), random forests, k-nearest-neighbours, and support-vector regressors. Every candidate is trained and scored the same way, and the results are reported even when a method flops. Negative results are kept on the books deliberately — the project's research log reads like a lab notebook, not a highlight reel.

One member has special status: **ordinary least squares (OLS) regression is the designated index**. Exactly like a passive benchmark in asset management, every fancier method must beat plain OLS on held-out data to justify its complexity. (Several don't.)

4.3 What a gradient-boosted tree is, in one paragraph

The workhorse models here (LightGBM, XGBoost, CatBoost) are **gradient-boosted decision trees**. A decision tree is a flowchart of yes/no questions ("has he averaged over 4 points in the last 5 games? is the opponent's defensive form weak?") ending in a numeric prediction. One tree is crude. Boosting builds hundreds of small trees **in sequence, where each new tree is fit to the errors the previous ones are still making** — a chain of specialists, each correcting the last. For tabular data like ours (rows $\times$ columns, no images or text), this family has dominated practical machine learning for a decade, which is why it displaced the neural network (LSTM) this project started with in its thesis era: sequence models need far more data per player than 38 gameweeks a season provides, while boosted trees pool learning across all players.

4.4 The median-versus-mean trap (the most important idea in this document)

Models are trained by minimizing a **loss function**, and the choice quietly decides **what kind of guess** the model makes. Squared-error loss produces the conditional **mean**; absolute-error (MAE) loss produces the conditional **median**. For symmetric bell-curve data those coincide. FPL points are wildly asymmetric: most weeks a player returns 2 points, occasionally he explodes for 15. Think of income distributions: the median UK salary is far below the mean because a few salaries are enormous.

Our best forecaster, CatBoost trained with MAE loss, therefore predicts something close to each player’s **median** week — and its forecasts sum to only about half the points actually scored across the league. It systematically ignores the explosive upside weeks. Is that bad? It depends what the number is used for, and this became the project’s central finding:

- For **ranking** players (“who is better this week?”), median-style forecasts are excellent — this model identifies the week’s top scorer better than any alternative we tested.
- For **absolute-scale decisions** (is a transfer worth its 4-point fee?), a forecast at half the true level distorts the exchange rate between points and fees.

We tested the obvious fix — rescaling each position’s forecasts so they sum correctly, with the scaling factor estimated on past data only — and it made the squads **worse** (details in Section 7). The lesson: a theoretically motivated correction is a hypothesis, not a fix, until the backtest agrees.

4.5 Combining models: a diversification story that failed, instructively

With twelve forecasters, the natural instinct is to blend them — diversification for models. The system estimated optimal blend weights (non-negative least squares) and for a long time the blend was the production forecaster.

Then a clean head-to-head experiment showed the **single best model beat the twelve-model blend at every position**. This is not a quirk; it is a classic result in the forecasting literature (Clemen 1989), and finance students already know its portfolio cousin: optimized (Markowitz) portfolio weights, estimated from finite noisy data, routinely lose out-of-sample to naive equal weighting, because estimation error in the weights swamps the theoretical gain from optimizing. Twelve highly correlated forecasters, weights estimated on a short window — the weights were fitting noise. We tested the standard remedies too (equal-weighting the top 3; ridge-regularized weights); the single best model still won.

So today, production is **one CatBoost model per position** — chosen, importantly, not by ideology but by a **bake-off** that re-runs on every training pass: single-best vs. three combination schemes, all scored on the same held-out weeks. If a future change ever makes a blend win again, production follows the evidence automatically.

5 How we know whether any of this works

5.1 Walk-forward evaluation: the honest backtest

All evaluation mimics real life: to forecast gameweek t , train only on gameweeks strictly before t ; predict; roll forward; repeat. No model, weight, scaling factor or hyperparameter is ever estimated on data from the period it is judged on. This is precisely how a quant fund should backtest a strategy, for precisely the same reason: any leak from the future produces a seductive number that evaporates in live trading.

5.2 The scorecards

- **MAE** (mean absolute error): on average, how many points off is each player-week forecast? Ours run around 0.85-1.05 depending on position.

- **MASE** (mean absolute scaled error): MAE divided by the error of a naive “he’ll score whatever he scored last week” forecast. Below 1.0 means you beat the naive rule. This is the headline accuracy metric because raw MAE is uninterpretable for a “bursty” series — like judging a demand forecaster without knowing how lumpy demand is. Our production models score roughly 0.5 (GK) to 0.85 (FWD).
- **Decision-aligned diagnostics:** calibration (do the forecasts sum to the points actually scored?), rank correlation (do we order players correctly within a week?), and **top-1 capture** (how often is our predicted best player actually the week’s best — the captaincy question). These exist because of the median-vs-mean trap: a model can improve MASE while becoming worse for decisions.
- **The gold standard: realized points.** Feed the forecasts through the optimizer over a full historical season and count the points the resulting squads actually scored. This is the only number that measures the whole system.

That last metric earns its “gold standard” title because — demonstrated twice now in this project — **forecast accuracy improvements do not reliably produce better squads**. A feature upgrade once improved MASE at every position while the realized-points backtest dropped from 1,966 to 1,880. Accuracy metrics are the compass; realized points are the terrain.

6 The optimizer

Given predicted points for every player over the next few weeks, squad selection becomes a **mixed-integer linear program** (MILP) — “integer” because you cannot hold 0.4 of a defender. It is the same mathematical species as classic portfolio optimization with lot-size constraints, and it can be solved **exactly**; no guesswork, no heuristics.

Maximize predicted points, subject to:

- budget (£100m), squad shape (2 GK / 5 DEF / 5 MID / 3 FWD), max 3 per club;
- a legal starting XI each week (formation rules), captain scoring double;
- transfer accounting: one free transfer a week, extras cost 4 points each;
- chips (wildcard, free hit, bench boost, triple captain), each usable once under FPL’s rules.

Because a whole season at once is computationally hopeless (and pointless anyway — forecasts degrade with distance), it solves on a **rolling horizon**: optimize weeks t through $t + 2$, commit only week t ’s decisions, roll forward, re-solve. Corporate planning under uncertainty works the same way.

A regression-proof detail: an automated test suite checks every optimizer output against all the rules above (budget, formation, club caps), so a code change that quietly produces illegal squads is caught by the tests, not by a mysterious backtest number.

7 What the numbers actually say

All figures below are realized points over the same 31-gameweek window (2024-25 season, weeks 1-31), same optimizer, so they are directly comparable:

System	Points	Comment
Thesis-era LSTM neural network	1526	the starting point
First Python rewrite (LightGBM)	1811	+19% from architecture change

System	Points	Comment
Model ensemble, longer history	1966	later found ~100 pts inflated by a leakage bug
Honest re-run: 12-model blend	1869	the clean baseline
Honest re-run: CatBoost only	1856	statistical tie with the blend, 1/12th the cost
CatBoost + level rescaling	1800	the “obvious fix” that failed

Three takeaways worth internalizing, because they generalize far beyond fantasy football:

1. **Most of the improvement came from process, not cleverness** — honest validation, leakage fixes, longer history, and letting a bake-off (not a preference) choose the model. The glamorous additions (37 new features in one batch; model blending; level recalibration) added roughly nothing or less than nothing.
2. **The metric you optimize is a design decision with consequences.** MAE-trained models predict medians; optimizers consume absolute scales; captaincy consumes tail behaviour. No single number captures all three, which is why the system now reports a dashboard rather than one score.
3. **Negative results are assets.** The failed recalibration, the failed blend, the accuracy-up/points-down episodes — each is logged with its numbers and mechanism in the research log, and each permanently changed how the next experiment is judged.

8 Where the project goes from here

In priority order, and all subject to the same rule — beat ~1,870 realized points honestly or be reported as a negative result:

- **Hyperparameter tuning** (running at the time of writing): every model so far uses hand-set configuration knobs; an automated search (Optuna) with time-ordered cross-validation is giving CatBoost a properly fair shot.
- **Upside-aware captaincy**: a parallel quantile model already estimates each player’s 90th-percentile week; feeding that into the captain choice attacks the tail question directly.
- **Component decomposition** (the big architectural bet): instead of predicting points as one number, predict its ingredients — minutes, goals, assists, clean sheets — and recombine. Clean sheets are a team-level event, so eleven defenders’ forecasts could share one team model, borrowing statistical strength the current per-player features cannot.
- **A live-season interface**: the weekly driver already refreshes data, retrains, and prints recommended transfers for a real team via the FPL API; a friendlier surface for it is future work once the 2026-27 season opens.

Companion documents: `report/main.typ` (the formal write-up with full methodology), `RESEARCH_LOG.md` (chronological lab notebook, including everything that failed), `TODO.md` (current open items), `CLAUDE.md` (architecture reference for the codebase).